1. Install and prepare

RoamScope reads Wi-Fi telemetry on your Windows laptop and creates local reports. This guide is for new users; no macOS helper, Git, GitHub Desktop or administrator shell is needed for normal operation.

Before you start

Use a Windows desktop/laptop with Wi-Fi enabled, a working Wi-Fi adapter/driver, and Python 3. Use an OS and Python version supported by their vendors. Exact minimum versions and all adapter combinations have not been certified. Internet access is needed to download the source, Python and optional plotting dependencies. On managed devices, use approved software and permissions.

Get the source

Open https://github.com/Half-Duplex-Life/roamscope and choose Code > Download ZIP, or use the supplied RoamScope.zip. Access requires authorization while the repository is private. Extract All into a new writable folder. Do not run from inside the ZIP. Open the folder containing roamscope.py, requirements.txt, Windows-Check.cmd and the roamscope subfolder; this may be nested inside the extracted archive.

Install Python and open PowerShell

Obtain a supported Python 3 release through https://www.python.org/downloads/windows/ or your approved software channel. Follow the current Python installation instructions. Close and reopen PowerShell after installation. In File Explorer, open the source folder, type powershell in its address bar and press Enter. Check:

```
py -3 --version
Get-Item .\roamscope.py, .\requirements.txt
```

If py is unavailable, try python --version and use python instead of py -3 in commands that create the environment. It must report Python 3; if it opens the Store or is not found, finish Python installation before proceeding.

Create an isolated environment

```
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

The second command installs Matplotlib for the original detailed graphs. If dependencies cannot be downloaded, the current app can still generate built-in SVG charts. The commands below call the environment interpreter directly, so PowerShell activation and execution-policy changes are unnecessary.

2. Check access and run a survey

Connect and run the preflight

Connect to the Wi-Fi network you intend to examine. Use one connected Wi-Fi adapter for a survey; multiple active adapters are deliberately rejected when ambiguous. Run:

```
.\.venv\Scripts\python.exe roamscope.py --doctor
```

Proceed only when the preflight reports a readable connected BSSID. A BSSID is the associated access point radio identity; it is not your laptop MAC address. RoamScope prefers Native Wi-Fi, then an English netsh fallback. It never fills missing association identity from neighbor scans.

Location permissions or access denied

Windows may restrict Wi-Fi identity access through location permissions. If preflight reports unavailable identity or access denied, inspect Settings > Privacy & security > Location on Windows 11, or Settings > Privacy > Location on Windows 10. Review the available device, app and desktop-app location controls and grant access only if appropriate. Labels and controls vary by Windows version and policy. Relaunch the terminal after approved changes. If controls are managed, ask your administrator; do not bypass policy or assume running as administrator fixes the issue.

First run: a 30-second stationary check

```
.\.venv\Scripts\python.exe windows_check.py --seconds 30
```

This runs preflight, disables neighbor scanning, and creates a separate timestamped output folder. With one AP and a stationary laptop, normally expect one stable BSSID and zero observed transitions. Double-clicking Windows-Check.cmd is an alternative five-minute check, but it uses system Python rather than necessarily your .venv; use the command above when you want the environment you configured.

A five-minute survey

```
.\.venv\Scripts\python.exe roamscope.py --duration 300 --interval 2 --csv --no-open
```

This normal survey samples about every two seconds and enables periodic neighbor scanning by default. To omit scans, add --scan-interval 0. For an authorized multi-AP test, walk between coverage areas on the same SSID. Keep the laptop awake. Never use --simulate to validate real Wi-Fi.

Stop manually or try a synthetic example

```
.\.venv\Scripts\python.exe roamscope.py --duration 0 --csv --no-open
# Press Ctrl+C once to stop and generate the report.
.\.venv\Scripts\python.exe roamscope.py --simulate --duration 30 --no-open
```

A finite duration is simplest for a first run. Do not close the terminal before it prints the report path.

3. Open and interpret your results

Locate the report

For normal surveys, open roamscope-out/report.html in File Explorer and choose your browser. For windows_check.py, use the report.html inside roamscope-out/windows-check-YYYYMMDD-HHMMSS/. The terminal prints the exact location. The report is self-contained; an internet connection is not needed to view it.

Files you will see

The timestamped .sqlite database stores samples and events. With --csv, three timestamped CSVs contain samples, observed roaming events and neighbor scans. report.html contains summary statistics, charts and associated AP details. Normal surveys reuse report.html in the same output folder, so it may replace an earlier report; the timestamped databases/CSVs remain. Use --out with a separate folder to preserve reports for each run.

What a successful stationary run shows

A valid connected BSSID for each sample, a stable associated AP, signal readings, available channel/band/frequency and transmit rate, plus an AP table and readable charts. Zero observed transitions is expected on a single AP. An empty scans CSV is expected when --scan-interval 0 was used. An empty roams CSV can be valid for a stationary run.

Unavailable metrics and limits

Windows RSSI is estimated from signal quality. Noise/SNR and channel width are not provided by the current Windows collector. Frequency/band may be unknown without trustworthy band evidence. Empty or unavailable fields are not measured zeros. SVG fallback charts are simpler than the optional Matplotlib charts. Missing BSSID means AP dwell and roaming detection are unavailable or incomplete, even if RSSI continues to appear.

A channel change alone does not prove an AP transition. Two-second polling can miss brief transitions; timestamps show sample observations. The sticky/fast labels are heuristics, not proof of 802.11r. Per-roam 802.11r negotiation and actual interruption duration are outside scope.

Validation status

A short Windows single-AP check recorded 10 samples over about 18 seconds, with valid BSSID in all samples using Native Wi-Fi. CSV/SQLite counts and generated HTML content were checked. Windows multi-AP roaming and browser display confirmation remain pending. The reported macOS pilot detected nine transitions aligned with nine Mist events within about 0-2 seconds. See VALIDATION.md for the current test matrix.

Share results carefully

windows_check.py creates shareable-summary.json with counts and availability checks, excluding network names and BSSIDs. Raw doctor output, CSVs, databases and HTML can contain network identifiers. Review and redact them before sharing. Keep local captures and reports out of GitHub, including public issues.

4. Troubleshooting and reference

Python, folders and packages

Python not found: finish installation, reopen PowerShell, and verify py -3 --version or python --version. File not found: open the extracted folder that actually contains roamscope.py. Missing packages: rerun installation with .\.venv\Scripts\python.exe -m pip install -r requirements.txt. If installation fails offline, use SVG fallback charts or an approved package source. Do not change system execution policy to run these commands.

No BSSID or WLAN telemetry

Check that Wi-Fi is enabled and connected, then compare netsh wlan show interfaces with --doctor. Do not confuse Physical address with AP BSSID. If Native Wi-Fi is denied, review permitted location access and WLAN AutoConfig service availability. A localized netsh fallback may not parse; the native API avoids localized labels. Record the exact error and Windows/adapter details. Do not guess a BSSID or substitute a scan result.

Report seems blank or does not open

Wait for the report path and open that exact report.html directly from the extracted output folder. Use a current browser and check that the file is nonempty. With no valid BSSID, expect an identity warning rather than valid roaming/AP results. Missing noise should display an unavailable message. If the whole page still appears blank, retain the report and console error locally and share a redacted description. Browser display has not been certified across all browsers.

Optional local dashboard

```
.\.venv\Scripts\python.exe roamscope.py --duration 300 --live --csv
```

Open http://127.0.0.1:8899/report.html while collection runs; the initial report may take a few seconds. The dashboard listens on loopback. If port 8899 is already in use, select another port with --port 8900 and open the matching address. There is no need to expose it through your router or firewall.

More commands

```
.\.venv\Scripts\python.exe roamscope.py --help
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

The test suite uses synthetic fixtures. It checks parsing and reporting; passing tests does not certify your adapter or prove real roaming. No Swift helper or packet-capture driver is required for the Windows workflow.

Official references

Python installation: https://docs.python.org/3/using/windows.html
Wi-Fi and location permissions: https://learn.microsoft.com/windows/win32/nativewifi/wi-fi-access-location-changes
Project: https://github.com/Half-Duplex-Life/roamscope

Guide updated 20 October 2026. The repository remains private until its owner explicitly approves public release. No open-source license has been selected; public visibility alone does not grant a reuse license.